Module 3 — RAG and Retrieval

This is the RAG-and-retrieval module of a knowledge base on production enterprise AI; it can be understood on its own.

Why this module exists: Retrieval is where most enterprise AI value — and most enterprise AI failure — actually lives. RAG, short for Retrieval-Augmented Generation, is the dominant pattern for giving a model current, private, permissioned, citable knowledge without fine-tuning. It's also the single best mitigation for hallucination on factual tasks. This module builds RAG from the ground up, then layers on the advanced techniques and the failure modes. Along the way it begins weaving in the theme of Systems-of-Record integration, often abbreviated SoR, a theme that continues through the systems-integration material later in this knowledge base.

Here's what this module covers. First, the retrieval problem and what RAG actually is. Then chunking strategies. Then embedding models for retrieval. Then vector databases, such as pgvector, Pinecone, and Weaviate. Then semantic versus hybrid search. Then reranking. Then advanced RAG, including query rewriting, multi-hop retrieval, contextual retrieval, and GraphRAG. Then Systems-of-Record integration, meaning grounding on live SoR data and entitlement-aware retrieval. And finally, retrieval evaluation and failure modes.

3.1 The retrieval problem, and what RAG is

Why it matters. Models have three knowledge gaps: they don't know your private data, they don't know anything after their training cutoff, and they hallucinate on specifics. RAG closes all three by retrieving relevant text at query time and putting it in the prompt, so the model answers from provided sources rather than from memory. It's the difference between asking "what do you remember?" and saying "here are the documents — answer from these and cite them."

Here's an analogy. A bare language model is like a closed-book exam: it recalls from memory and makes confident guesses. RAG is like an open-book exam: it looks up the relevant page, answers from it, and cites it. The open-book approach is more accurate, more updatable, and more auditable.

How it works. The classic RAG pipeline, from Lewis and colleagues in 2020, generalized, has four stages.

First, ingest, done offline: collect documents, chunk them, embed each chunk, and store the vectors plus text plus metadata in a vector database.

Second, retrieve, done online: embed the user's query, find the top-k most similar chunks through semantic search, and optionally rerank them.

Third, augment: insert the retrieved chunks into the prompt with instructions such as "answer only from this context, cite sources, and say you don't know if the answer isn't supported."

Fourth, generate: the model produces a grounded, cited answer.

Why RAG beats fine-tuning for knowledge. RAG is updatable in real time — you change the data, not the model. It's permissionable per user, because you can filter what's retrieved. It's citable and auditable. It's cheaper. And it doesn't bake stale facts into weights.

When to use it (and when not to). Use RAG whenever the answer depends on data that is private, changing, large, or must be cited or permissioned — which is to say, nearly all enterprise knowledge work. Skip RAG when the task is pure reasoning or transformation over text that's already in the prompt.

On the question "is RAG dead?" — the long-context debate. Million-token windows and agents did not replace retrieval. For large, changing, permissioned, citable corpora, retrieval remains far cheaper and more auditable than context-stuffing — widely cited as roughly an order of magnitude cheaper, with better latency and traceability. The mid-2026 shift in mental model is toward what people call "context engineering": retrieval is one tool among several, chosen often through adaptive routing, alongside long context, tools, and memory, for assembling the right context per query.

Where it goes wrong. RAG is easy to demo and hard to make reliable. Most failures are retrieval failures — the wrong or missing chunks — not generation failures. You can't reason well over the wrong context. Garbage retrieved leads to garbage answered, but confidently.

A concrete example. Consider the question "What's our return policy for enterprise hardware?" A bare model invents a plausible policy. A RAG system retrieves the actual policy document section and answers from it with a citation — and returns "I couldn't find that" if nothing relevant exists.

3.2 Chunking strategies

Why it matters. Documents are too big to embed or retrieve whole, so you split them into chunks. Chunking quality caps the quality of everything downstream. Chunk too big and retrieval is imprecise and dilutes attention. Chunk too small and you sever the context needed to understand a passage. It's the most under-appreciated lever in RAG.

How it works. Here are the strategies, ordered from weakest to strongest for most cases.

Fixed-size chunking, for example 500 tokens, with overlap, for example 10 to 20 percent — a simple baseline, where the overlap avoids cutting mid-idea. It's blind to structure.

Recursive, or character, splitting — split on a hierarchy of separators, from paragraphs to sentences to words, to respect natural boundaries. This is a common default.

Structure-aware chunking — split on document structure, such as markdown headings, HTML sections, PDF layout, code functions, or table rows. Much better, because chunks align to semantic units.

Semantic chunking — detect topic shifts, by measuring embedding similarity between sentences, and split there. Better coherence, but more compute.

Sentence-window, or small-to-big, chunking — retrieve on small precise chunks, but feed the model a larger surrounding window for context. A strong pattern.

Parent-document, or hierarchical, retrieval — index small child chunks, but return the parent section. This balances precision and context.

Contextual chunking — prepend a short, LLM-generated summary of the chunk's place in the document before embedding, so isolated chunks stay interpretable. This connects to the contextual retrieval technique covered later.

Metadata is part of chunking. Store with each chunk: the source document, the section, the date, the author, access-control tags and entitlements, which are critical for the Systems-of-Record work, and any structured fields you'll filter on.

The main options for tooling. LangChain and LlamaIndex splitters, the `unstructured` library for layout-aware parsing of PDFs and Office documents, Docling, and vendor ingestion pipelines.

When to use it (and when not to). Match chunking to document type. For structured documents, use structure-aware chunking. For dense prose, use semantic or recursive chunking. For contracts and policies, use section-aware chunking with generous overlap so clauses aren't severed. For tables, use row- or record-oriented chunking. Here's a defensible default to deviate from: start with recursive splitting at roughly 300 to 500 tokens with about 15 percent overlap for prose, then measure and adjust. Go smaller, roughly 150 to 250 tokens, when you need precise pinpoint retrieval and will use small-to-big or parent-document expansion. Go larger, roughly 800 to 1,000 tokens, for narrative documents where continuity matters. Tune against your embedding model's sweet spot and measure through retrieval evaluation. There is no universal chunk size — but "recursive, roughly 400 tokens, roughly 15 percent overlap" is a sane starting point.

Where it goes wrong. Severed context — a clause split from its heading. For instance, "Section 7.2, the party may terminate" without knowing which party or which contract.

Mixing topics in one chunk, which produces muddy embeddings and poor retrieval.

Losing tables and layout — naive text extraction destroys tabular meaning, so use layout-aware parsers.

No metadata — then you can't filter by recency or entitlement later, which is a governance dead-end.

One-size chunking across heterogeneous corpora.

A concrete example. For master service agreements, the team chunks by clause and section, prepends the contract title and section heading to each chunk, which is the contextual approach, and tags each chunk with the owning account and its access group. That way retrieval is precise and also entitlement-filterable.

3.3 Embedding models for retrieval

Why it matters. The embedding model decides what "similar" means for your corpus. Its quality directly determines whether the right chunks are retrievable at all. Choosing and evaluating it is a first-class decision, not a default.

How it works — the selection axes.

Retrieval quality on your domain — measure with your own evaluation set. Public leaderboards, such as MTEB, are a starting point, not an answer.

Dimensionality — higher dimensions can capture more but cost more in storage and latency. Matryoshka embeddings let you truncate dimensions for a tunable tradeoff.

Maximum input length — it must accommodate your chunk size.

Multilingual needs.

Multimodality and long context, which is a newer axis — several current models, including Cohere Embed v4, Jina v4, and Gemini, embed text and also images and PDF pages into one space, which is useful for charts, tables, and scans. And very long context, with Embed v4 handling roughly 128,000 tokens, relaxes the chunk-size ceiling.

Query versus document asymmetry — some models use different encodings for queries versus passages, or instruction prefixes like "query" and "passage." Use them correctly.

Domain adaptation — general embeddings can miss jargon, whether medical, legal, telecom, or product SKUs. Domain-tuned or fine-tuned embeddings help.

Hosting and residency — API versus self-hosted. Open models like bge, e5, nomic, gte, and Qwen3-Embedding support data-residency and compliance needs.

The main options among vendors, as of mid-2026. Google Gemini Embedding, model name gemini-embedding-001, and the open-weight Qwen3-Embedding lead on MTEB. Cohere Embed v4 — multimodal, roughly 128,000-token context, with Matryoshka plus int8 and binary compression — and Voyage-3.5 and its lite variant, with domain variants, are strong. OpenAI's text-embedding-3-large is still solid but no longer

benchmark-leading. On the open side: Qwen3, Jina v4, which is multimodal, plus BGE, E5, GTE, and Nomic. The open-versus-proprietary gap has largely closed. Note, and this needs a verify flag: MTEB rankings drift monthly — verify.

When to use it (and when not to). Pick 2 or 3 candidates, run your own retrieval evaluation, and choose on quality-per-cost. Prefer a model you can afford to re-run over the whole corpus when you upgrade. If data can't leave your boundary, choose a strong open model you can self-host.

Where it goes wrong. Embedding the query and the documents with different models, which breaks the geometry. Ignoring instruction prefixes the model expects. Assuming biggest equals best. And forgetting that changing the model means re-indexing everything.

A concrete example. A pharma company finds a general embedding model conflates drug names. A domain-tuned biomedical embedding model lifts retrieval hit-rate on their evaluation set from 71 percent to 89 percent, and they self-host it to keep documents on-premises.

3.4 Vector databases

Why it matters. A vector database stores embeddings and finds the nearest ones to a query vector fast, at scale. This is Approximate Nearest Neighbor search, often abbreviated ANN. It's the retrieval engine of RAG. The market ranges from "a Postgres extension" to dedicated distributed systems, and the right choice depends on scale, your existing stack, and your filtering needs.

How it works.

Why "approximate"? Exact nearest-neighbor means comparing the query to every stored vector — that's linear per query, which is far too slow at millions of vectors. Approximate Nearest Neighbor search pre-builds an index, a navigable "map" of the vector space, that you traverse greedily to find almost the closest vectors while touching only a tiny fraction of them. It trades occasional misses for a speedup of 100 to 1000 times.

ANN indexes trade a little recall for big speed. HNSW is a hierarchical navigable small-world graph — think of layered shortcuts, coarse-to-fine, that you hop across to home in on neighbors. It's low-latency but memory-heavy, and it's the most common. IVF and IVF-PQ cluster the space, then search only the nearest clusters, using product quantization to compress vectors for memory savings. DiskANN is an SSD-backed graph for huge corpora that won't fit in RAM.

Metadata filtering restricts search by fields, such as date, source, or entitlement tags. How filtering interacts with the ANN index — pre-filter versus post-filter — affects both correctness and speed. Entitlement

filtering must be pre-filter and correct, never best-effort. Filtered-ANN "under-fetching" was a classic failure; pgvector version 0.8's iterative index scans specifically address it.

Quantization — scalar, product, or binary — shrinks vectors for cost, trading some recall.

Hybrid support — many stores now index both dense vectors and sparse or keyword signals, such as BM25, for hybrid search.

The main options, and how they differ.

pgvector, a Postgres extension, keeps vectors inside your existing Postgres. Its huge advantage is that you co-locate them with your relational data, get transactional consistency and familiar operations, join vectors to business tables, and reuse row-level security. It's a pragmatic default for many enterprises up to large scale, though it may need tuning or partitioning at very high scale. pgvector version 0.8 added iterative index scans plus faster HNSW build and query, and pgvectorscale adds StreamingDiskANN plus Statistical Binary Quantization for compressed, disk-backed scale.

Pinecone is fully managed, serverless, scales easily, and has strong metadata filtering. It's SaaS, so data leaves your VPC unless you use private networking tiers, and its cost is usage-based.

Weaviate is open-source or managed, with built-in hybrid search, modules, and GraphQL. It's self-hostable for residency.

Qdrant is open-source, performant, with strong filtering. You can self-host or use it managed.

Milvus and Zilliz are built for very large scale, with multiple index types.

Elasticsearch and OpenSearch offer mature keyword search plus added vector support. They're great when you already run them and want hybrid.

Cloud-native options include Azure AI Search, Vertex AI Vector Search, MongoDB Atlas Vector Search, and Redis. Their appeal is convenience and integration with your existing platform.

When to use it (and when not to). If you're already on Postgres, need transactional consistency and joins to business data, and are at moderate scale, choose pgvector — often the pragmatic winner, with fewer moving parts. If you're at massive scale and want zero operations, with SaaS acceptable, choose Pinecone or Zilliz. If you need data residency, must self-host, or want built-in hybrid, choose Weaviate, Qdrant, or Milvus. If you already run Elastic or OpenSearch, use its vector support for hybrid. The decisive factors are scale, latency SLA, filtering correctness — especially entitlements — hybrid support, hosting and residency, and operational burden.

Where it goes wrong. Recall versus speed misconfigured — over-aggressive ANN or quantization silently drops relevant results.

Filtering that's post-hoc — entitlement filters applied after ANN can miss or leak, so ensure correct pre-filtering, which is security-critical.

Stale index — the source data changes but the vectors don't; you need re-index or change-data-capture pipelines.

Over-engineering — spinning up a distributed vector cluster when pgvector would do, which adds operational cost and a sync problem.

Metadata as an afterthought — you can't filter or permission what you didn't store.

A concrete example. A company already running managed Postgres for its CRM-adjacent data uses pgvector. Chunks live in the same database as the accounts they belong to, entitlement tags are enforced with Postgres row-level security, and retrieval joins vectors to live account rows. That's one system, one security model, and transactional consistency.

3.5 Semantic versus hybrid search

Why it matters. Semantic, or dense, search matches by meaning, using embeddings. Keyword, or sparse or lexical, search — for example BM25 — matches by exact terms. Each fails where the other shines. Hybrid search combines them and is, in practice, the strongest general default for enterprise retrieval.

How it works.

Semantic search is great for paraphrase, synonyms, and concepts — "end my plan" is roughly equivalent to "cancel subscription." It's weak on exact identifiers.

Keyword search, such as BM25, is great for exact tokens: product SKUs, error codes, part numbers, names, acronyms, legal citations — precisely where embeddings blur. For instance, "SKU-4471-B" must match exactly.

Hybrid search runs both, then fuses the two result lists. The standard method is Reciprocal Rank Fusion, or RRF. It scores each result by the reciprocal of its rank position in each list. The formula is one divided by (k plus the rank position), summed across lists, where k is a small constant, and the results are re-sorted by that sum. The key insight is that RRF uses only rank positions, not raw scores. That's exactly why it sidesteps the problem that BM25 scores and cosine similarities live on incomparable scales. Weighted score-blending is the alternative, but it requires normalizing those scales first. Hybrid captures both meaning and exact matches.

When to use it (and when not to). For corpora full of identifiers, codes, names, and jargon — the enterprise reality of SKUs, incident IDs, and config codes — use hybrid. For pure conceptual prose,

semantic may suffice. Because enterprise data is riddled with exact identifiers, default to hybrid unless you've measured that semantic-only is enough.

Where it goes wrong. Semantic-only missing exact SKU or code matches, which is a classic CPQ failure. Keyword-only missing paraphrased questions. And naive score fusion, since raw cosine and BM25 scores aren't comparable — use RRF or normalize.

A concrete example. A user asks for "pricing for the enterprise firewall bundle SKU FW-9000." Semantic search finds the firewall and bundle concepts. Keyword search nails "FW-9000." Hybrid returns the exact product page and related bundles. Semantic-only would risk returning the wrong SKU's price — dangerous in CPQ.

3.6 Reranking

Why it matters. Initial retrieval optimizes for speed at scale — ANN over millions of chunks — so it's approximate. A reranker takes the top 20 to 100 candidates and re-scores them for true relevance with a heavier, more accurate model, keeping only the best few for the prompt. It's one of the highest-ROI additions to a RAG pipeline: better precision, fewer tokens, and less of the "lost in the middle" problem.

How it works.

Two-stage retrieval, and why it's two stages. A bi-encoder embeds the query and each document separately, so document vectors can be pre-computed once at index time and cached — that's fast and scalable, but the model never sees query and document together. A cross-encoder feeds the query and document as one input and scores their interaction — far more accurate, but it can't be pre-computed, because it needs the query, so every query-document pair is a fresh model call. That caching asymmetry is the whole reason for the design: use the cheap, cacheable bi-encoder to retrieve a candidate set over millions of chunks, then spend the expensive cross-encoder only on the 20 to 100 survivors.

LLM rerankers prompt a language model to score or order passages — more powerful, more expensive, and overlapping with the LLM-as-judge idea.

The output is a tightly relevant, ordered short list, for example the top 3 to 5, placed in the prompt, best-first.

The main options among vendors, as of mid-2026. Cohere Rerank 4.0, in fast and pro variants, and 3.5, which is multilingual, handles JSON and semi-structured data, and takes 4096 tokens. Voyage rerank-2.5 and its lite variant, with code, finance, and legal variants. Jina Reranker v2, which is open, multilingual, and multimodal. The open bge-reranker-v2-m3. Newer entrants, such as Zerank. And Elastic and Vertex

rerankers plus vendor-native reranking in RAG platforms. On typical English RAG, the top options cluster within a few nDCG points — so pick on latency, cost, language, and your own evaluation.

When to use it (and when not to). Add reranking when retrieval recall is decent but precision is poor — the right documents are somewhere in the top 50 but not the top 5 — or when you must minimize context tokens for cost or latency, or when "lost in the middle" is hurting answers. It's nearly always worth testing.

Where it goes wrong. Added latency and cost per query, so budget for it. Reranking can't fix bad recall — if the right chunk isn't in the candidate set, reranking can't rescue it, so fix chunking, embeddings, or hybrid first. And over-trimming to top-1 can drop needed context.

A concrete example. A support RAG bot retrieves 50 candidate knowledge-base passages through hybrid search, reranks to the 4 most relevant, and feeds those to the model. That cuts prompt tokens by roughly 80 percent while actually raising answer accuracy, because the model now sees only on-point context.

3.7 Advanced RAG

Beyond "chunk, embed, retrieve," these techniques handle hard queries, multi-step questions, and structured knowledge. Maturity varies, and it's labeled per technique.

Query rewriting and transformation. Maturity: Established, moving toward Stabilizing. The user's raw question is often a poor search query — ambiguous, pronoun-laden, or multi-part. A language model rewrites or expands it before retrieval. Query expansion adds synonyms and related terms. Decomposition splits a compound question into sub-queries and retrieves each. HyDE, which stands for Hypothetical Document Embeddings, generates a hypothetical answer, embeds that, and retrieves — which often matches real documents better than the terse question. And conversational rewriting resolves something like "what about the enterprise tier?" into a standalone query using chat history. A pitfall: rewriting can drift from intent, so keep the original as a fallback.

Multi-hop retrieval. Maturity: Stabilizing, becoming mainstream via agentic RAG. Some questions need chained lookups. For example, "Which of our accounts renewing this quarter use the discontinued module?" means finding renewals, then finding module usage, then intersecting the two. The system retrieves, reasons, then retrieves again — an agentic RAG loop, where the model decides what to fetch next. As of 2026 this is a mainstream best-practice frame, not fringe. It's more powerful, but it carries more latency and cost and is harder to evaluate.

Adaptive RAG. Maturity: Stabilizing. A classifier routes each query to the cheapest sufficient pipeline. Trivial questions are answered directly, moderate ones through single-shot RAG, and hard ones through multi-hop or agentic retrieval. This is increasingly the default way to balance cost and quality.

Contextual retrieval. Maturity: Established; popularized by Anthropic in 2024. Before embedding each chunk, prepend a short, LLM-generated description of the chunk's context within its document — for example, "This is from ACME's 2025 MSA, Section 7, on termination." This fixes the "severed context" problem from the chunking discussion and materially improves retrieval accuracy. Anthropic reported roughly 35 percent fewer retrieval failures, and more when combined with hybrid search and reranking. It's now offered as a built-in feature in managed RAG platforms, such as Amazon Bedrock Knowledge Bases.

GraphRAG. Maturity: Stabilizing; the cost barrier is now largely solved. Instead of, or alongside, vector search over text, you build a knowledge graph from the corpus and retrieve over entities and relationships. Here's the mechanic that makes it different, and how it answers a "global" question. Microsoft's GraphRAG extracts entities and relations from the whole corpus, runs community detection to cluster related entities into groups, and pre-generates a summary of each community. A global question like "what themes span all our incident reports?" is then answered by map-reducing over those community summaries — aggregating across the entire corpus. Vector RAG, by contrast, can only fetch the top-k local passages and literally never sees the whole picture. GraphRAG is also strong for multi-hop reasoning over structured relationships. As for tradeoffs: classic GraphRAG's up-front graph construction is expensive and can be noisy, with entity-resolution errors. But LazyGraphRAG, from Microsoft and open-source, defers summarization to query time using cheap noun-phrase co-occurrence. This cuts indexing cost to roughly that of vector RAG — about 0.1 percent of full GraphRAG — while keeping strong global-question quality, which largely removes the adoption blocker. It's usually combined with vector RAG, an approach called hybrid GraphRAG, routed by query type, rather than replacing it.

Other patterns worth naming. Self-RAG and corrective RAG, or CRAG — the model critiques retrieved context and re-retrieves or abstains if it's insufficient, which reduces answering from irrelevant context. And fusion retrieval — multiple query variants plus result fusion.

When to use it (and when not to). Add complexity only when basic RAG's evaluation shows a specific gap. For ambiguous queries, add rewriting. For compound questions, add decomposition or multi-hop. For isolated-chunk failures, add contextual retrieval. For global or relational questions, add GraphRAG. Each addition costs latency, money, and evaluability — so earn it.

A concrete example. A revenue-operations analyst asks, "Across all at-risk enterprise accounts, what common product gaps are driving churn?" Vector RAG returns scattered passages. GraphRAG over an account-product-issue graph surfaces the recurring relationships and summarizes the theme — a global question that basic RAG can't answer well.

3.8 Systems-of-Record integration — grounding on live SoR data and entitlement-aware retrieval

This is a cross-cutting theme, and here we cover the read paths. This is where RAG meets the enterprise's authoritative data. The full treatment — write paths, the semantic layer, platform-native versus composable approaches, and a CPQ walkthrough — comes in the later systems-integration material. Here we establish the retrieval side.

The core pattern. Enterprise knowledge doesn't live only in documents; it lives in Systems of Record, or SoR — CRM such as Salesforce, ITSM and workflow such as ServiceNow, ERP, and data warehouses. The durable architecture is "AI as the interaction and orchestration layer, the SoR as the system of record." For retrieval, that means the agent grounds its answers in live SoR data, not a stale copy, and treats the SoR, not the model, as the source of truth.

How it works — two grounding modes.

Structured retrieval, meaning you query the SoR directly. For authoritative, current, transactional facts — an opportunity's amount, a quote's status, an incident's priority — the agent uses tool calls, or an MCP server, to query the SoR API live. This is often better than embedding such data, because it must be exact and current. For example, a call such as "get opportunity by id" on the CRM returns the real, up-to-the-second record.

Unstructured retrieval, meaning vector or hybrid RAG. For knowledge articles, notes, attachments, past cases, and contracts, the classic RAG pipeline applies, indexing SoR-attached content.

Real systems combine both: retrieve the structured record and the related unstructured context, then reason.

Entitlement-aware and row-level-security-aware retrieval — the non-negotiable. The agent must see only what the requesting user is permitted to see. An AI layer that bypasses SoR permissions is a data-exfiltration incident waiting to happen — a rep querying accounts they don't own, or a customer seeing another customer's data. Here are the mechanisms.

Propagate the user's identity end-to-end, not a service account with god-mode. The retrieval layer acts as the user.

Enforce entitlements at the source where possible — call the SoR API with the user's context so the SoR's own sharing rules and ACLs apply, such as the Salesforce sharing model or ServiceNow ACLs. This is the safest approach, because the authoritative system decides.

For vector stores, tag every chunk with access-control metadata at ingest, and pre-filter retrieval by the user's entitlements. This must be correct-by-construction, never best-effort. A post-hoc filter that occasionally leaks is a breach.

Re-check at generation and action time, because permissions can change; high-stakes reads and writes re-validate.

And never let the language model be the access-control boundary. The model can be prompt-injected; authorization lives in code and in the SoR.

When to use it (and when not to). Query the SoR live for anything that must be exact, current, or transactional. Use vector RAG for unstructured knowledge. Always enforce entitlements at the authoritative source, and pre-filter vector retrieval by access tags.

Where it goes wrong. Stale mirror — copying SoR data into a vector store and answering from a day-old snapshot, showing wrong prices or closed deals shown as open. Prefer live queries for volatile facts, and use change-data-capture to keep mirrors fresh.

Entitlement bypass — the RAG index ignores sharing rules, so users see forbidden records. This is the number one enterprise RAG security failure.

Embedding volatile transactional data — amounts and statuses change constantly, so embedding them guarantees staleness.

Confusing "retrieved" with "authoritative" — retrieved text can be an old draft, so tie answers to the SoR's current record and cite it.

A concrete example. A sales rep asks the assistant, "What's the status of the ACME renewal, and are there open support issues?" The agent, acting as the rep, calls Salesforce for the opportunity and quote — live, entitlement-filtered by sharing rules — and calls ServiceNow for open incidents, filtered by ACLs. It also vector-retrieves the account's recent call notes, tagged to accounts the rep can access. Then it answers with citations, showing nothing the rep isn't allowed to see. The write path — actually updating that renewal — comes in the later systems-integration material.

3.9 Retrieval evaluation and failure modes

Why it matters. Most RAG failures are retrieval failures, and most teams can't see them because they only look at final answers. Retrieval evaluation decomposes the pipeline so you know where it breaks — retrieval versus generation — and can fix the right thing. This is the applied version of the general evaluation material, specialized to RAG.

How it works — what to measure.

Retrieval metrics, which need labeled sets mapping each query to its relevant chunks. Recall@k, or hit rate — is the right chunk in the top k? If not, generation is doomed. Precision@k, MRR, and nDCG — how

highly ranked and how clean are the results? And context precision and recall, from RAG frameworks — the fraction of retrieved context that's relevant, and the fraction of needed information that was retrieved.

Generation-given-context metrics. Faithfulness, or groundedness — is every claim supported by the retrieved context? This detects hallucination despite retrieval. Answer relevance — does it address the question? And citation accuracy — do citations point to text that actually supports the claim?

End-to-end metrics — task success, correctness versus gold answers, and human or LLM-judge scores.

And the RAG "triad," in the TruLens framing: context relevance, then groundedness, then answer relevance. If all three are high, the system is working, and each one localizes a different failure.

The main options for tooling. RAGAS, TruLens, DeepEval, promptfoo, Arize Phoenix, LangSmith, and vendor evaluation suites. Build a golden set of real queries with known-relevant sources and expected answers, and grow it from production failures.

When to use it — the diagnostic flow. First, is recall@k low? Then fix chunking, embeddings, add hybrid search, or add query rewriting. Second, is recall good but precision low or token count high? Then add reranking. Third, is the context good but the answers unfaithful? Then fix the prompt — "answer only from context," and cite — lower the temperature, or recognize the model isn't following context and consider a stronger model or Self-RAG. Fourth, is everything good offline but users are unhappy? Then your golden set isn't representative; expand it from real traffic.

Where it goes wrong — the field guide. Missing content — the answer isn't in the corpus at all, so the system must say "I don't know," not fabricate.

Retrieval miss — it's in the corpus but not retrieved, due to bad chunking, bad embeddings, or no hybrid search.

Lost in the middle — retrieved but buried or ignored, so rerank, trim, and reorder best-first.

Ignored context — the model answers from parametric memory despite good context, which is a prompt or model issue.

Unfaithful synthesis — the model blends context with invented details, so use groundedness evaluation and citations.

Wrong chunk, right topic — it retrieved a related-but-incorrect passage, for example the wrong SKU's price, so use hybrid search, reranking, and metadata filters.

Entitlement leak — it retrieved data the user shouldn't see. This is a security failure, caught by access-control tests, not by quality metrics.

Stale data — the mirror is out of sync with the SoR, so use change-data-capture or live queries.

Conflicting sources — the corpus contains contradictory versions, so dedupe, prefer the authoritative source, and surface the conflict.

A concrete example. A CPQ RAG assistant gives a wrong discount answer. Evaluation reveals recall@10 is fine — the right policy chunk is retrieved — but groundedness is low, because the model is merging two discount tiers. The fix is reranking to surface the single relevant tier first, plus a prompt that forces per-claim citation. Separately, an access-control test catches that the index would have returned another region's pricing to an unauthorized rep — fixed by pre-filtering on entitlement tags.

Module 3 takeaways

RAG means retrieving relevant text at query time and answering from it — the primary fix for private, current, and citable knowledge, and for hallucination.

Chunking caps everything downstream, so be structure-aware and metadata-aware.

Embeddings plus a vector database are the engine. pgvector is a pragmatic default — co-located, transactional, and row-level-security-friendly — with Pinecone, Weaviate, Qdrant, and others for scale or residency.

Hybrid search, combining semantic and keyword, is the enterprise default because your data is full of exact identifiers. And reranking is high-ROI precision.

Advanced RAG — query rewriting, multi-hop, contextual retrieval, and GraphRAG — earns its complexity only against a measured gap.

For Systems-of-Record integration: ground on live SoR data for volatile facts, use RAG for unstructured knowledge, and make retrieval entitlement-aware at the authoritative source — never let the language model be the access boundary.

Evaluate retrieval separately from generation. Most failures are retrieval failures, and entitlement leaks are security failures that your quality metrics won't catch.